

Otimização de alocação de ativos e posicionamento de equipes de manutenção

Matheus Cascalho dos Santos
Engenharia de Controle e Automação
Email: matheuscascalho@ufmg.br

Pedro Goulart
Engenharia de Controle e Automação
Email: pedrohegoulart@gmail.com

Abstract—This paper addresses the optimization of asset allocation and the positioning of maintenance teams in the context of mining dam monitoring. The problem is formulated as two mono-objective optimization model, minimizing the total distances traveled by maintenance teams and balancing workload distribution among them. The proposed approach combines a constructive heuristic to generate the initial solution, leveraging the K-Means clustering algorithm, and a Variable Neighborhood Search (VNS) metaheuristic with a first-improvement refinement strategy to explore the solution space. Results demonstrate the sensitivity of the algorithm to initial solutions and its ability to improve poorly initialized solutions effectively. However, the algorithm occasionally converges to local optima, highlighting the need for further enhancements to escape such scenarios. The findings underscore the importance of a robust initial solution and a balanced refinement strategy in addressing complex optimization problems in industrial contexts.

I. INTRODUÇÃO

O monitoramento e a manutenção de barragens de mineração são cruciais para garantir a segurança estrutural e prevenir desastres ambientais e sociais. Com a implementação de regulamentações rigorosas a partir de 2010, as empresas foram obrigadas a instalar sistemas automatizados, como sirenes, sensores e medidores, para alertar as zonas de autossalvamento (ZAS) em caso de emergências. Contudo, a ampliação desses sistemas resultou em um aumento significativo no número de ativos que demandam manutenção regular, exigindo uma abordagem eficiente para alocação de recursos.

Nesse contexto, surge a necessidade de um plano de otimização para alocação de equipes de manutenção em bases predefinidas e para atribuição dos ativos às equipes responsáveis. O objetivo é minimizar as distâncias totais percorridas e equilibrar a carga de trabalho entre as equipes, respeitando restrições como: cada equipe deve ser alocada a uma única base; cada ativo deve ser atribuído a apenas uma base e equipe; e a distribuição de ativos entre as equipes deve ser proporcional à sua capacidade.

A solução para esse problema de otimização não apenas assegura maior eficiência operacional, mas também contribui para a confiabilidade dos sistemas de segurança, reduzindo tempos de resposta e garantindo a continuidade do monitoramento. A seguir, este artigo apresenta a formulação matemática do problema, os métodos de solução e as principais análises realizadas.

II. MODELAGEM MATEMÁTICA

A. Variáveis

- $x_{ij} \in \{0, 1\}$: 1 se o ativo i for atribuído à base j ; 0, caso contrário;
- $y_{ij} \in \{0, 1\}$: 1 se a base j for ocupada pela equipe k ; 0, caso contrário;
- $h_{ij} \in \{0, 1\}$: 1 se o ativo i for mantido pela equipe k ; 0, caso contrário.

B. Parâmetros

- d_{ij} : representa a distância entre o ativo i e a base de manutenção j .

C. Conjuntos

- N : Ativos;
- M : Bases;
- S : Equipes.

D. Funções Objetivo

1) *Função objetivo f_1* : O objetivo é minimizar a soma das distâncias entre os ativos e suas respectivas equipes de manutenção, ou seja, minimizar o custo total de deslocamento. Matematicamente, isso é expresso por:

$$\min f_1 = \sum_{i=1}^n \sum_{j=1}^m d_{ij} x_{ij}$$

2) *função Objetivo f_2* : Esta função busca minimizar a diferença de carga de trabalho entre as equipes de manutenção, ou seja, a diferença entre o número de ativos mantidos pela equipe mais sobrecarregada e pela equipe menos sobrecarregada. Isso é expresso por:

$$\min f_2 = \max_k \left(\sum_{i=1}^n h_{ik} \right) - \min_k \left(\sum_{i=1}^n h_{ik} \right)$$

E. Restrições

- 1) Cada equipe deve ser alocada a exatamente uma base de manutenção:

$$\sum_{j=1}^M y_{jk} = 1 \quad \forall k \in S$$

- 2) Cada ativo deve ser atribuído a exatamente uma base de manutenção:

$$\sum_{j=1}^M x_{ij} = 1 \quad \forall i \in N$$

- 3) Cada ativo só pode ser atribuído a uma base se esta estiver ocupada por pelo menos uma equipe de manutenção:

$$x_{ij} \leq y_{jk} \quad \forall i \in N \quad \forall j \in M \quad \forall k \in S$$

- 4) Cada ativo deve ser atribuído a exatamente uma equipe de manutenção:

$$\sum_{k=1}^S h_{ik} = 1 \quad \forall i \in N$$

- 5) Cada ativo só pode ser atribuído a uma equipe se esta estiver ocupando a base na qual o ativo estiver alocado:

$$h_{ik} \leq \frac{x_{ij} + y_{jk}}{2} \quad \forall i \in N \quad \forall j \in M \quad \forall k \in S$$

- 6) Cada equipe deve ser responsável por pelo menos $\eta \frac{n}{s}$ ativos, onde $\eta = 0.2$ é um percentual definido pela empresa:

$$\sum_{i=1}^N h_{ik} \geq \eta \frac{n}{s} \quad \forall i \in N \quad \forall k \in S$$

III. ALGORITMO

A. Estrutura de dados da Solução

Observamos pelas restrições 1, 2 e 4 que, se definirmos um ativo a determinada equipe, e se equipe para uma base, então consequentemente sabemos à que base esse ativo está vinculado. Dessa forma, a variável x pode ser calculada a partir das variáveis y e h . Além disso, como cada equipe só pode estar em uma base, e cada ativo só pode estar vinculado à uma equipe, podemos definir as variáveis y e h como vetores unidimensionais preenchidos com valores inteiros de forma que:

- h é um vetor onde cada posição representa um ativo e o valor de cada posição representa o índice da equipe que está vinculada com esse ativo.
- y é um vetor onde cada posição representa uma equipe e o valor de cada posição representa o índice da base que está vinculada com essa equipe.

A figura 1 apresenta a implementação dessa solução utilizando uma estrutura do Python chamada `dataclass` [1]. Os vetores x , y e h foram renomeados respectivamente para **ativo_base**, **equipe_base** e **ativo_equipe**, a fim de facilitar a leitura do código. Como já explicado, o vetor x (método **ativo_base**) é calculado a partir dos outros vetores utilizando um laço **for** que itera sobre o vetor h para obter a equipe de cada ativo e obtém a partir do vetor y a base dessa equipe.

Ao utilizar essa estrutura de dados, eliminamos a necessidade de implementar as restrições 1, 2 e 4, uma vez que elas já são respeitadas naturalmente pela estrutura de dados. O método **ativo_base** também elimina a necessidade de implementar as restrições 3 e 5, pois elas já serão respeitadas pela forma com que o vetor x será construído.

```

34 @dataclass  # Matheus Cascalho *
35 class solution:
36     ativo_equipe: list # h
37     equipe_base: list # y
38     fitness: float = 0
39     penalidade: float = 0
40
41     @property  # Matheus Cascalho
42     def fitness_penalizado(self):
43         return self.penalidade + self.fitness
44
45     @property  # Matheus Cascalho *
46     def ativo_base(self): # x
47         x = []
48         for equipe in self.ativo_equipe:
49             base = self.equipe_base[equipe]
50             x.append(base)
51         return x

```

Fig. 1. Estrutura de dados que modela a solução do problema

B. Tratamento de restrições

Ao contrário das demais, a restrição 6 não é resolvida diretamente pela estrutura de dados da solução. Portanto, para a solução do problema, iremos modelar essa restrição como uma função de penalidade somada à função objetivo. Para isso, devemos primeiramente converter a restrição no formato padrão de uma restrição de desigualdade:

$$g(h) = \eta \frac{n}{s} - \sum_i h_{ik} \leq 0 \quad \forall i \quad \forall k$$

Dessa forma, a função de penalidade pode então ser modelada da seguinte forma:

$$P(h) = u \max(0, g(h))^2$$

Definimos $u = 100$ como constante de penalidade na solução do problema.

C. Vizinhanças

Definimos 4 estruturas de vizinhança, que geram soluções vizinhas a partir da variação dos ativos de cada equipe ou das bases de cada equipe. Essas vizinhanças são descritas no algoritmo 1, que apresenta o pseudocódigo da função `shake` utilizada no algoritmo de busca.

As estruturas 1 e 2 são estruturas que geram pequenas perturbações na solução atual, de forma que é útil para encontrar ótimos locais. Já as estruturas 3 e 4 geram grandes perturbações, permitindo que o algoritmo escape de ótimos locais em busca de novas regiões ótimas.

D. Algoritmo de Busca

Como algoritmo de busca da solução ótima, utilizamos uma versão da heurística **BasicVNS** que utiliza como estratégia de refinamento o método **first_improvement**. A escolha dessa estratégia de refinamento se deve ao fato de que o espaço de busca é muito grande e variado, o que faria com que uma busca extensiva em todo o espaço de vizinhança fosse caro

Algorithm 1: shake

Input: Solução Atual x , Índice de Vizinhaça k ,
Problema $prob_def$
Output: Nova Solução y

```
1  $y \leftarrow$  Cópia de  $x$ ;  
2  $r\_equipe \leftarrow$  Índice Aleatório de Equipe;  
3  $r\_ativos \leftarrow$  Permutação Aleatória de Ativos;  
4 if  $k == 1$  then  
5   Alterar a base de  $r\_equipe$ ;  
6 else if  $k == 2$  then  
7   Alterar até 10 ativos aleatoriamente;  
8 else if  $k == 3$  then  
9   Alterar a base de  $r\_equipe$ ;  
10  Alterar até 30 ativos aleatoriamente;  
11 else if  $k == 4$  then  
12  Alterar todas as bases e até 30 ativos  
    aleatoriamente;  
13 return  $y$ ;
```

computacionalmente. Os algoritmos 2, 3 e 4 apresentam o pseudocódigo desses métodos.

O método **Basic VNS** utilizado tem como critério de parada a verificação de estabilidade das soluções. Essa estabilidade é verificada da seguinte forma: Se um histórico de soluções possui mais de 200 soluções (esse valor é parametrizável) e os últimos 20% das soluções não apresentaram variação na função objetivo penalizada, então o algoritmo chegou a uma solução estável. Esse critério foi adotado a fim evitar uma quantidade desnecessária de iterações. Entretanto, essa abordagem pode impedir que o algoritmo escape de mínimos locais. Por esse motivo, aumentamos o mínimo de soluções para 40 mil durante a validação dos resultados.

O método **first_improvement** faz uma varredura na vizinhança e retorna a primeira solução vizinha que possui um resultado melhor que a solução atual. Dado a grande quantidade de vizinhos que algumas estruturas vizinhanças possuem, definimos um máximo de vizinhos que serão avaliados para cada estrutura por meio do parâmetro **max_neighbors**.

O método **neighborhoodChange** utilizado é o mesmo encontrado na literatura, e seu pseudocódigo é descrito no algoritmo 4.

E. Heurística Construtiva da solução inicial

Para construir a solução inicial do algoritmo de busca, analisamos a distribuição geográfica dos ativos. A figura 2 apresenta essa distribuição, onde é possível observar 3 regiões de concentração dos ativos. Dessa forma, uma boa solução inicial é definir a base mais próxima do ponto médio de cada região como a base de atendimento dos ativos dessa região. Para implementar essa heurística utilizamos a biblioteca **scikit learn** [2] para segregar os ativos e bases em 3 clusters e identificar a posição do centróide de cada cluster. A solução inicial portanto define a posição inicial das equipes na base

Algorithm 2: BasicVNS

Input:
Problema $prob_def$,
Solução Inicial $initial_solution$,
Função Objetivo $objective_function$,
Iterações Máximas $max_iteration$,
Histórico $historico$,
Máximo de Vizinhaças $kmax$
Output: Histórico de Soluções

```
1  $it \leftarrow 0$ ;  
2  $current\_solution \leftarrow initial\_solution$ ;  
3 while  $it \leq max\_iteration$  do  
4    $k \leftarrow 1$ ;  
5   while  $k \leq kmax$  do  
6      $new\_solution \leftarrow first\_improvement()$ ;  
7     Avaliar  $new\_solution$  com  
         $objective\_function$ ;  
8     Atualizar  $it$ ;  
9     Atualizar  $(current\_solution, k)$  com  
         $neighborhoodChange()$ ;  
10    Adicionar  $current\_solution$  ao histórico;  
11    if  $historico.is\_stable$  then  
12      break;  
13 return  $historico$ ;
```

Algorithm 3: first_improvement

Input: Solução Atual x , Índice de Vizinhaça k ,
Função Objetivo $objective_function$,
Problema $prob_def$, Iterações Máximas
 $max_iteration$
Output: Solução Melhorada

```
1  $current\_fitness \leftarrow$  Fitness Penalizado de  $x$ ;  
2  $neighbor\_fitness \leftarrow \infty$ ;  
3  $it \leftarrow 0$ ;  
4 while  $neighbor\_fitness > current\_fitness$  and  
    $it < max\_iteration$  do  
5    $neighbor \leftarrow shake(x, k, prob\_def)$ ;  
6    $neighbor\_fitness \leftarrow$  Fitness Penalizado de  
    $neighbor$ ;  
7   Incrementar  $it$ ;  
8   if  $it > max\_neighbors[k]$  then  
9      $neighbor \leftarrow x$ ;  
10    break;  
11 return  $neighbor$ ;
```

mais próxima a esses centróides e define os ativos desses clusters a essas equipes.

Algorithm 4: neighborhoodChange

Input: Solução Atual x , Nova Solução y , Índice de Vizinhaça k

Output: Solução Atualizada x , Novo Índice de Vizinhaça k

```

1 if Fitness Penalizado de  $y < x$  then
2    $x \leftarrow$  Cópia de  $y$ ;
3    $k \leftarrow 1$ ;
4 else
5   Incrementar  $k$ ;
6 return  $(x, k)$ ;

```

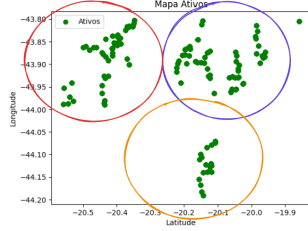


Fig. 2. Agrupamento de ativos em clusters

IV. RESULTADOS

A. Resultados com a Heurística Construtiva

A figura 3 apresenta o gráfico de convergência do algoritmo otimizando a partir da solução inicial gerada pela heurística construtiva do centróide do cluster. Como é possível observar, o algoritmo não conseguiu encontrar nenhum resultado melhor que a solução inicial, ficando estável durante todos os testes em todas as iterações.

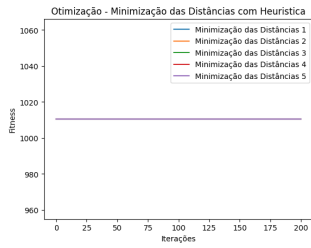


Fig. 3. Gráfico de Convergência do algoritmo para a função f_1 : Minimização de distância utilizando heurística construtiva de centróides

A fim de avaliar a qualidade do algoritmo no que diz respeito a busca de solução ótima, iremos avaliar o resultado partindo de uma solução inicial gerada aleatoriamente.

B. Resultados com solução inicial aleatória

A figura 4 apresenta o gráfico de convergência das soluções em cada um dos 5 testes realizados para a otimização da função objetivo f_1 . As estatísticas desses testes são apresentadas na tabela I.

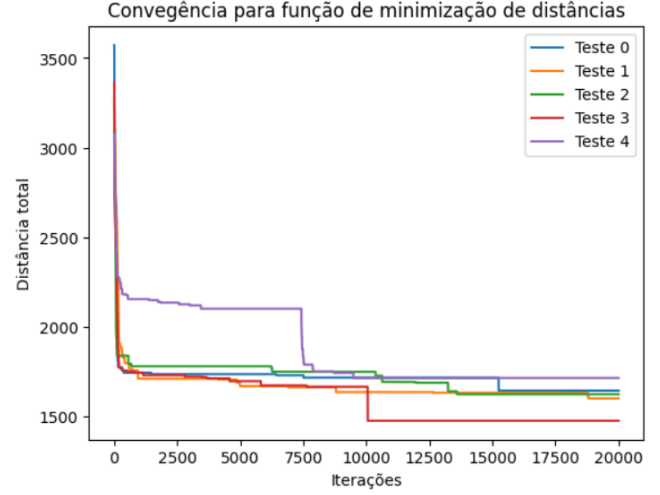


Fig. 4. Gráfico de Convergência do algoritmo para a função f_1 : Minimização de distância

TABLE I

RESULTADOS ESTATÍSTICOS DA SOLUÇÃO ÓTIMA ENCONTRADA A CADA TESTE.

min	1476.033
max	1714.75
std	77,90

Observamos pela figura 4 que o algoritmo utilizado se mostrou muito eficiente para escapar de soluções iniciais ruins, uma vez que todas as curvas tiveram uma redução da distância muito rápida nas iterações iniciais. Entretanto, o algoritmo ficou preso em alguns ótimos locais por muitas iterações, como pode ser visto na curva do teste 4, que mostra um grande platô até a iteração 7500. Também observamos que o algoritmo de busca é muito sensível à solução inicial utilizada, uma vez que cada teste chegou a uma solução ótima distinta. O resultado da melhor solução ótima encontrada para a função objetivo f_1 é apresentado na figura 5.

A figura 6 apresenta o gráfico de convergência da função f_2 . Ao contrário da função anterior, o algoritmo conseguiu encontrar a solução ótima muito rápido, convergindo para o mesmo resultado em todas os testes. Vale observar entretanto que cada solução encontrada possui um resultado diferente, como pode ser observado na tabela II, que apresenta o vetor y (equipe_base) de cada solução ótima encontrada

TABLE II

SOLUÇÕES ENCONTRADAS PARA A FUNÇÃO f_2

Teste 0	[6, 10, 10]
Teste 1	[10, 9, 5]
Teste 2	[11, 6, 2]
Teste 3	[0, 6, 3]
Teste 4	[1, 13, 5]

O resultado da uma das soluções ótimas encontradas para a função objetivo f_2 é apresentado na figura 7

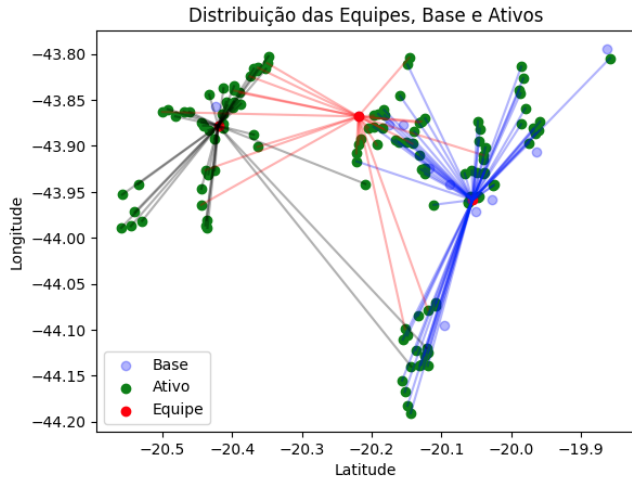


Fig. 5. Resultado a partir de solução inicial aleatória de otimização da função f_1

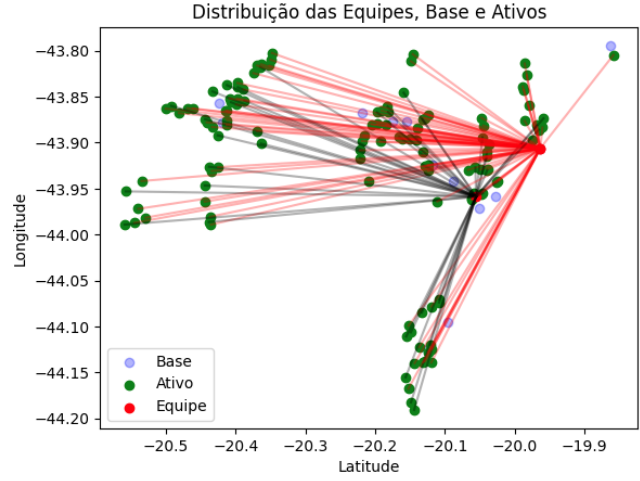


Fig. 7. Resultado a partir de solução inicial aleatória de otimização da função f_2

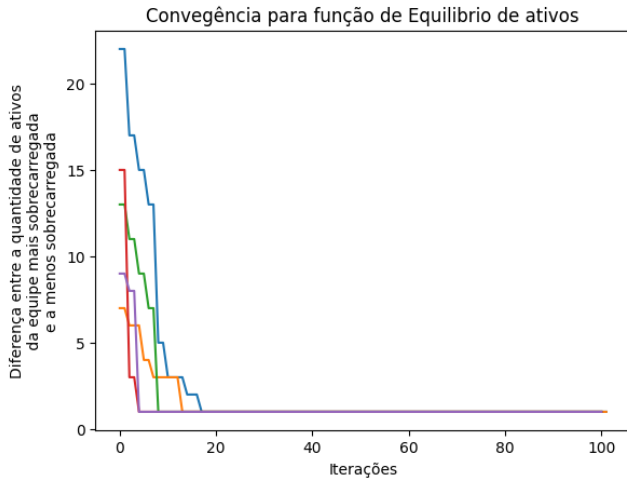


Fig. 6. Gráfico de Convergência do algoritmo para a função f_2 : Equilíbrio de Ativos

V. CONCLUSÃO

Neste trabalho, foi apresentada uma abordagem para a otimização da alocação de ativos e posicionamento de equipes de manutenção, considerando as peculiaridades do monitoramento de barragens de mineração. A modelagem do problema como dois problemas de otimização mono-objetivo permitiu minimizar os custos de deslocamento e equilibrar a carga de trabalho entre as equipes, atendendo às restrições operacionais.

A solução inicial, gerada por uma heurística construtiva baseada em agrupamento geográfico (K-Means), mostrou-se adequada para identificar uma configuração inicial próxima do ótimo. Contudo, os resultados revelaram que o algoritmo VNS apresenta forte sensibilidade à qualidade da solução inicial, destacando a importância de estratégias de diversificação mais robustas para escapar de ótimos locais. A aplicação da estratégia de refinamento *first-improvement* se mostrou eficiente em reduzir custos nas iterações iniciais, mas limitada em alcançar

soluções globais em casos onde o algoritmo ficava preso em platôs.

No geral, a metodologia demonstrou potencial para resolver problemas complexos de alocação no setor industrial, mas requer melhorias nas estratégias de exploração e intensificação do espaço de busca. Trabalhos futuros podem explorar métodos híbridos ou ajustes adaptativos nas estruturas de vizinhança para aumentar a robustez do algoritmo. Por fim, os resultados obtidos destacam a relevância de integrar técnicas heurísticas e metaheurísticas para solucionar problemas reais com múltiplos critérios e restrições.

REFERENCES

- [1] S. Chodarev and S. Ilyas, "Metamodel-based language definition with python," 2023.
- [2] O. Kramer and O. Kramer, "Scikit-learn," *Machine learning for evolution strategies*, pp. 45–53, 2016.